



UNIVERSITY
OF APPLIED SCIENCES
UPPER AUSTRIA

Game Engine Architectures

Fachhochschule Oberösterreich

C++ Basics

Basics

C++ Arrays

- Arrays store **multiple values of the same type** in a single variable
- Must be declared with **square brackets [] and the size**
- The number of dimensions is, in theory, unlimited
- The first element has **index 0**
- We have to manage the **size of arrays** ourselves!

Basics

C++ Arrays

```
int main()
{
    string students[4];
    string myStudents[4] = {"Hannes", "Susanne", "Florian", "Maria"};
    int numbers[4];

    numbers[0] = 1;
    numbers[1] = 2;
    numbers[2] = 3;
    numbers[3] = 4;

    cout << numbers[3] << endl;
    cout << numbers[4] << endl;
}
```

What is the output of this program?

4

-2384902 (some memory content)

Basics

C++ Arrays

Browse to:

<https://onecompiler.com/cpp/3zq3pxcxh>

Main.cpp

HGB_Arrays 

NEW

CPP ▼

RUN ▶

⋮

⌵

```
1 #include <iostream>
2 using namespace std;
3
4 int main()
5 {
6
7     int fields[3][3];
8
9     for (int i = 0; i < 3; i++)
10         for(int j=0; j<3; j++)
11             fields[i][j] = 0; // TODO: fill array with values from 0-8
12
13     for (int i = 0; i < 3*3; i++)
14         ; // TODO: fill array with values from 0-8
15
16     for (int i = 0; i < 3; i++)
17         for (int j = 0; j < 3; j++)
18             cout << fields[i][j] << endl;
19
20     return 0;
21 }
```

STDIN

Input for the program (Optional)

Output:

0
0
0
0
0
0
0
0
0

Basics

C++ Arrays

```
for (int i = 0; i < 3; i++)  
    for(int j=0; j<3; j++)  
        fields[i][j] = (i*3)+j;
```

Simple math – we jump i columns (width = 3) and add j

Basics

C++ Arrays

```
for (int i = 0; i < 3*3; i++)  
    fields[i/3][i%3]=i;
```

Simple math – we need to find the column value ($i/3$)
and the row value ($i\%3$)
Example: $5/3 = 1$, $5\%3 = 2 \Rightarrow \text{fields}[1][2]$

Basics

C++ Arrays

```
int main()
{
    int fields[9];

    for (int i = 0; i < 3*3; i++)
        fields[i]=i;

    int x=1;
    int y=2;

    //cout << fields[?] << endl;

    return 0;
}
```

Simulate with a 1-dimensional array

Basics

C++ Arrays

```
int main()
{
    int fields[9];
    cout << sizeof(fields) << endl;
}
```

What if we forget how big the array is?

We can ask using “sizeof”

What is the output?

36

We get the results in bytes, an int = 4 bytes

Basics

C++ Arrays

```
int main()
{
    int fields[9];

    cout << sizeof(fields)/sizeof(int) << endl;
}
```

What if we forget how big the array is?
We can ask using "sizeof"
What is the output?
9

Basics

C++ Char Arrays

- Native C++ does not support strings out of the box
- However, strings are natively supported in form of character arrays

```
char myString[] = "Hello, Hagenberg";
```

- Char arrays are “null terminated“, i.e. end with char ,\0‘
- Beware, this only applies to string literals but not all char arrays!

```
char myString2[] = { 'H', 'e', 'l', 'l', 'o', '\0' };
```

In contrast to the example above, this is an “ordinary” array and not a string literal – consequently, you must null-terminate yourself!

Basics

C++ Strings

- The string library allows “real” strings (`#include <string>`)
- Can be concatenated using math operators (“Hallo” + “Hagenberg”)
- The length can be assessed using the `length()` function
- Special characters (such as “) must be escaped with \“

Basics

C++ Strings

Browse to:

<https://onecompiler.com/cpp/3zq59qweq>

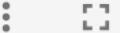
Main.cpp

HGB_Strings 

NEW

CPP ▾

RUN ▶



```
1 #include <iostream>
2 #include <string>
3 using namespace std;
4
5 int main()
6 {
7     string str = "Hallo";
8     str = str + " Hagenberg";
9
10
11     cout << str << endl;
12     int i = str.length();
13
14     cout << i << endl;
15
16     int j = 0;
17     //iterate over the string and print it character per character
18     //while(...)
19     {
20         // cout <<
21
22     }
23
24     return 0;
25 }
```

STDIN

Input for the program (Optional)

Output:

Hallo Hagenberg

15

```
int j = 0;  
while(str[j]!='\0')  
{  
    cout << str[j];  
    j++;  
}
```

As long as the current character in the string array is not the terminator (, \0')
We print this character and increment the counter

Basics

C++ References

- Alternative name for an existing object (“**Alias**“)
- A reference is not an object itself, it **points to an existing object**
- Must be initialized with an existing object
- Declaration using type&
- References **cannot be changed!**
- They act like a “**constant pointer**“ to the initialized variable

```
int main()
{
    int x = 1;
    int y = 2;
    int& ref = x;
}
```

Basics

C++ References

What is the output?
x=1 y=2 ref=1

```
int main()
{
    int x = 1;
    int y = 2;
    int& ref = x;

    cout<<"x="<<x<<" y="<<y<<" ref="<<ref<<"\n";

    ref = y;
    cout<<"x="<<x<<" y="<<y<<" ref="<<ref<<"\n";

    ref = 3;
    cout<<"x="<<x<<" y="<<y<<" ref="<<ref<<"\n";
}
```

Basics

C++ References

What is the output?
x=2 y=2 ref=2
The reference cannot be
changed, but the value can!

```
int main()
{
    int x = 1;
    int y = 2;
    int& ref = x;

    cout<<"x="<<x<<" y="<<y<<" ref="<<ref<<"\n";

    ref = y;
    cout<<"x="<<x<<" y="<<y<<" ref="<<ref<<"\n";

    ref = 3;
    cout<<"x="<<x<<" y="<<y<<" ref="<<ref<<"\n";
}
```

Basics

C++ References

What is the output?
x=3 y=2 ref=3

```
int main()
{
    int x = 1;
    int y = 2;
    int& ref = x;

    cout<<"x="<<x<<" y="<<y<<" ref="<<ref<<"\n";

    ref = y;
    cout<<"x="<<x<<" y="<<y<<" ref="<<ref<<"\n";

    ref = 3;
    cout<<"x="<<x<<" y="<<y<<" ref="<<ref<<"\n";
}
```

Basics

C++ References: Call by Reference

```
int countup(int x)
{
    x = x + 1;
    return x;
}
```

```
int countupRef(int& x)
{
    x = x + 1;
    return x;
}
```

What is the output?

1
2

What is the output?

2
2

```
int main()
{
    int i = 1;
    int j = countup(i);
    cout << i << endl;
    cout << j << endl;

    i = 1;
    j = countupRef(i);
    cout << i << endl;
    cout << j << endl;
}
```

Basics

C++ References: Call by Reference

→ References are mostly (if not always) used for “call by reference”

→ Advantages

- We can change the value of a parameter
- It saves memory and increases performance (no duplicate stored)
- It is more readable (i.e., the function header tells us what may happen)

→ Disadvantages

- We may need to check if the parameter is null
- Multithreading can become an issue

```
UFUNCTION(BlueprintImplementableEvent)  
void OnErrorMessageString(const FString& msg);
```

Example from our Unreal Project

Game Engine Architectures



UNIVERSITY
OF APPLIED SCIENCES
UPPER AUSTRIA

FH Oberösterreich

University of Applied Sciences Upper Austria

Digital Media Department

Media Technology & Design

Digital Arts

Interactive Media



Dr. Andreas Riegler

Professor of Interactive Systems

Digital Media Department

E Andreas.Riegler@fh-hagenberg.at